Gate 05 — DRF API

Status: Complete **Preconditions:** Gate 03 complete **Unlocks:** Gate 07 (Next.js Public Frontend)

Purpose

Build all DRF endpoints that the Next.js public frontend and any future coordinator frontend will consume. Every endpoint in this gate is the authoritative backend contract — field names, filter parameters, and response shapes defined here must not change without a versioning decision.

Tier-aware serializers are the primary access control mechanism at the API layer. Serializers filter fields based on `request.user.access_tier`; the `TierPermission` class (Gate 03) gates endpoint access entirely.

Endpoints

Method	Path	Min Tier	Description
GET	`/api/v1/species/`	1 (public)	Species list with filtering and search
GET	`/api/v1/species/{id}/`	1 (public)	Species detail with assessments and common names
GET	`/api/v1/institutions/`	1 (public)	Institution list
GET	`/api/v1/institutions/{id}/`	1 (public)	Institution detail
GET	`/api/v1/populations/`	3	Ex-situ population list (per-institution detail)
GET	`/api/v1/populations/{id}/`	3	Ex-situ population detail with holding records
GET	`/api/v1/field-programs/`	1 (public)	Field program list
GET	`/api/v1/field-programs/{id}/`	1 (public)	Field program detail
GET	`/api/v1/dashboard/`	1 (public)	Conservation summary statistics
GET	`/api/v1/map/localities/`	1 (public)	Species localities as GeoJSON FeatureCollection
GET	`/api/v1/map/watersheds/`	1 (public)	Watershed list (no geometry; for filters and info panels)
GET	`/api/v1/map/summary/`	1 (public)	Map aggregate statistics

All endpoints are read-only at MVP. Write operations go through Django Admin (Gate 04).

User Stories

BE-05-1: Species List

As a public user or Next.js frontend, **I want** `GET /api/v1/species/` to return a paginated, filterable species list, **so that** I can render the species directory.

Query parameters:

- `taxonomic_status` — filter by `described` / `undescribed_morphospecies` / `species_complex` / `uncertain`
- `iucn_status` — filter by IUCN category code (e.g., `CR`, `EN`)
- `family` — filter by family name
- `cares_status` — filter by CARES category (`CCR`, `CEN`, `CVU`, `CLC`)
- `endemic_status` — filter by `endemic` / `native` / `introduced`
- `search` — full-text search on `scientific_name`, `provisional_name`, `commonname__name`
- `page`, `page_size` — pagination (default page_size: 50, max: 200)
- `ordering` — `scientific_name` (default), `-iucn_status`, `family`

Response (200 OK):

```
{
  "count": 97,
  "described_count": 79,
  "undescribed_count": 18,
  "next": "/api/v1/species/?page=2",
  "previous": null,
  "results": [
    {
      "id": 1,
      "scientific_name": "Bedotia sp. 'manombo'",
      "taxonomic_status": "undescribed_morphospecies",
      "provisional_name": "'manombo'",
      "family": "Bedotiidae",
      "genus": "Bedotia",
      "endemic_status": "endemic",
      "iucn_status": "CR",
      "cares_status": "CCR",
      "shoal_priority": true,
      "common_names": [{"name": "Manombo Rainbowfish", "language": "en"}]
    }
  ]
}
```

Note: `authority`, `year_described`, and ecology/description fields are not included in the list response — only in detail (BE-05-2). `described_count` and `undescribed_count` are totals for the unfiltered dataset, used to render the directory header.

Acceptance Criteria:

Given a GET to `/api/v1/species/` with no parameters **When** the database contains 79 described and 18 undescribed species **Then** 97 results are returned; `described_count = 79`; `undescribed_count = 18`;

results are paginated at 50 per page

Given a GET to `/api/v1/species/?taxonomic_status=undescribed_morphospecies` **When** the filter is applied **Then** only undescribed morphospecies records are returned; `count` reflects the filtered set

Given a GET to `/api/v1/species/?search=manombo` **When** the search is applied **Then** "Bedotia sp. 'manombo'" appears in results (matched via `provisional_name` or `scientific_name`)

Given a GET to `/api/v1/species/?iucn_status=CR` **When** the filter is applied **Then** only CR-status species are returned; undescribed taxa with no IUCN assessment are excluded from this filter result (not shown under "Not Evaluated" when `iucn_status` filter is active)

BE-05-2: Species Detail

As a public user or Next.js frontend, **I want** `GET /api/v1/species/{id}/` to return the full species profile, **so that** I can render an individual species page.

Response (200 OK):

```
{
  "id": 1,
  "scientific_name": "Pachypanchax sakaramyi",
  "taxonomic_status": "described",
  "provisional_name": null,
  "authority": "Holly, 1928",
  "year_described": 1928,
  "family": "Aplocheilidae",
  "genus": "Pachypanchax",
  "endemic_status": "endemic",
  "iucn_status": "EN",
  "cares_status": "CEN",
  "shoal_priority": false,
  "description": "...",
  "ecology_notes": "...",
  "distribution_narrative": "...",
  "morphology": "...",
  "max_length_cm": "8.5",
  "habitat_type": "streams",
  "iucn_taxon_id": 166478,
  "common_names": [
    {"name": "Sakaramy killifish", "language": "en"},
    {"name": "Killi de Sakaramy", "language": "fr"}
  ],
  "conservation_assessments": [
    {
      "category": "EN",
      "source": "iucn_official",
      "assessment_date": "2016-10-01",
      "assessor": "IUCN SSC Freshwater Fish Specialist Group",
      "criteria": "B1ab(iii)",
      "review_status": "accepted"
    }
```

```
  ],
  "field_programs": [
    {"id": 3, "name": "Durrell Nosivolo", "status": "active"}
  ],
  "ex_situ_summary": {
    "institutions_holding": 4,
    "total_individuals": 120,
    "breeding_programs": 2
  }
}
```

Tier visibility rules:

Field	Tier 1–2	Tier 3+
`conservation_assessments`	Only `review_status = 'accepted'` records; `review_status` field omitted	All records; `review_status` and `review_notes` included
`ex_situ_summary`	Aggregate counts only (institutions_holding, total_individuals, breeding_programs)	Full per-institution detail available via `/api/v1/populations/`
`authority` / `year_described`	Shown (or null for undescribed taxa)	Same

Acceptance Criteria:

Given an anonymous GET to `/api/v1/species/{id}/` for a described species **When** the species has one `iucn_official` assessment with `review_status = 'accepted'` and one `recommended_revision` with `review_status = 'pending_review'` **Then** only the `iucn_official` assessment appears in `conservation_assessments`; the pending review is not visible

Given a Tier 3 GET to `/api/v1/species/{id}/` **When** the same species has both assessments **Then** both assessments appear; the pending review includes `review_status`, `review_notes`, `flagged_by`, `flagged_date`

Given a GET to `/api/v1/species/{id}/` for an undescribed morphospecies **When** the species has `taxonomic_status = 'undescribed_morphospecies'` **Then** `authority` and `year_described` are null; `provisional_name` is populated; the response does not omit these fields (they are present with null values)

Given a GET to `/api/v1/species/9999/` where no such species exists **When** the request is processed **Then** HTTP 404 is returned

---

BE-05-3: Institution List and Detail

As a public user, **I want** `GET /api/v1/institutions/` to return a list of institutions, **so that** I can see which organizations are involved in Madagascar fish conservation.

Query parameters: `institution_type`, `country`, `search` (name, city)

Tier visibility rules:

Field	Tier 1–2	Tier 3+
`name`, `institution_type`, `country`, `city`, `website`	Visible	Visible
`contact_email`, `species360_id`	Hidden (field omitted)	Visible

Acceptance Criteria:

Given an anonymous GET to `/api/v1/institutions/` **When** the response is returned **Then** `contact_email` is not present in any result object

Given a Tier 3 GET to `/api/v1/institutions/{id}/` **When** the institution has a `contact_email` set **Then** `contact_email` is present in the response

---

BE-05-4: Ex-Situ Population List and Detail (Tier 3+)

As a Tier 3 conservation coordinator, **I want** `GET /api/v1/populations/` to return per-institution captive population records, **so that** I can coordinate between holding institutions.

DRF endpoint: `GET /api/v1/populations/` **Min Tier:** 3 **Query parameters:** `species_id`, `institution_id`, `breeding_status`

Response (200 OK):

```
{
  "count": 45,
  "results": [
    {
      "id": 12,
      "species": {"id": 1, "scientific_name": "Pachypanchax sakaramyi"},
      "institution": {"id": 7, "name": "Cologne Zoo", "country": "Germany"},
      "count_total": 18,
      "count_male": 7,
      "count_female": 8,
      "count_unsexed": 3,
      "breeding_status": "breeding",
      "studbook_managed": false,
      "last_census_date": "2026-02-15"
    }
  ]
}
```

`GET /api/v1/populations/{id}/` additionally includes:

```
{
  "holding_records": [
    {"date": "2026-02-15", "count_total": 18, "notes": ""},
    {"date": "2025-11-01", "count_total": 24, "notes": "mortality event"}
  ]
}
```

Acceptance Criteria:

Given a Tier 2 GET to `/api/v1/populations/` **When** the request is processed **Then** HTTP 403 is returned

Given a Tier 3 GET to `/api/v1/populations/?species_id=1` **When** multiple institutions hold that species **Then** all ExSituPopulation records for that species are returned (read is not institution-scoped; only writes are)

BE-05-5: Field Program List and Detail

As a public user, **I want** `GET /api/v1/field-programs/` to return a list of active field programs, **so that** I can see what in-situ conservation work is underway.

Response: list with `id`, `name`, `description`, `lead_institution` (name + country), `region`, `status`, `start_date`, `focal_species` (list of scientific_name + iucn_status), `partner_institutions` (list of names)

Acceptance Criteria:

Given an anonymous GET to `/api/v1/field-programs/` **When** the database contains 3 active programs and 1 completed program **Then** all 4 are returned (no status filtering by default); the list is orderable by `status` and `name`

BE-05-6: Conservation Dashboard

As the Next.js conservation dashboard, **I want** `GET /api/v1/dashboard/` to return summary statistics, **so that** I can render the ex-situ coverage gap visualization without making N separate API calls.

Response (200 OK):

```
{
  "species_counts": {
    "total": 97,
    "described": 79,
    "undescribed": 18,
    "by_iucn_status": {
      "EX": 0, "EW": 0, "CR": 18, "EN": 22, "VU": 9,
      "NT": 4, "LC": 3, "DD": 12, "NE": 29
    }
  },
  "ex_situ_coverage": {
    "threatened_species_total": 49,
    "threatened_species_with_captive_population": 18,
    "threatened_species_without_captive_population": 31,
    "institutions_active": 23,
    "total_populations_tracked": 67
  },
  "field_programs": {
    "active": 3,
    "planned": 1,
    "completed": 2
  },
```

```
  "last_updated": "2026-04-12T10:00:00Z"
}
```

"Threatened" = IUCN status in `CR`, `EN`, `VU`.

This endpoint is cache-invalidated when any `Species`, `ExSituPopulation`, or `ConservationAssessment` record changes. Use Django signals to invalidate a Redis cache key; serve cached response otherwise.

Acceptance Criteria:

Given a public GET to `/api/v1/dashboard/` **When** the database has 49 threatened species and 18 have at least one ExSituPopulation record **Then** `threatened_species_with_captive_population = 18` and `threatened_species_without_captive_population = 31`

Given a new ExSituPopulation record is created for a previously uncovered CR species **When** `GET /api/v1/dashboard/` is called **Then** `threatened_species_with_captive_population` increases by 1 (cache invalidated by signal)

BE-05-7: Species Localities GeoJSON Endpoint

As the Next.js map page, **I want** `GET /api/v1/map/localities/` to return a GeoJSON FeatureCollection of species localities, **so that** I can render locality markers on the Leaflet map.

New dependency: Add `djangorestframework-gis` to `requirements.txt`. Add `rest_framework_gis` to `INSTALLED_APPS`.

Query parameters:

- `species_id` (integer) — filter to a single species
- `family` (string) — filter by family name
- `iucn_status` (string) — filter by IUCN category code
- `watershed_id` (integer) — filter by Watershed FK
- `locality_type` (string) — filter by locality type enum
- `presence_status` (string) — filter by presence status enum
- `coordinate_precision` (string) — filter by coordinate precision enum
- `bbox` (string) — bounding box: `min_lng,min_lat,max_lng,max_lat`

Response (200 OK):

```
{
  "type": "FeatureCollection",
  "features": [
    {
      "type": "Feature",
      "geometry": {
        "type": "Point",
        "coordinates": [47.52, -18.91]
      },
      "properties": {
        "id": 42,
        "species_id": 7,
        "scientific_name": "Pachypanchax sakaramyi",
```

```
        "family": "Aplocheilidae",
        "iucn_status": "EN",
        "locality_name": "Sakaramy River near Joffreville",
        "locality_type": "type_locality",
        "presence_status": "present",
        "water_body": "Sakaramy River",
        "water_body_type": "river",
        "drainage_basin_name": "Sambirano",
        "year_collected": 1928,
        "coordinate_precision": "exact",
        "source_citation": "Holly, 1928"
      }
    }
  ]
}
```

Tier behavior at MVP: All requests are Tier 1 (unauthenticated). The serializer serves `location_generalized` for records where `is_sensitive = True`, and `location` for non-sensitive records.

Serializer: `SpeciesLocalityGeoSerializer` using `rest_framework_gis.serializers.GeoFeatureModelSerializer`. Custom `get_geometry` method implements the tier-aware coordinate selection.

Performance:

- Use `select_related('species', 'drainage_basin')` to avoid N+1 queries
- Use `only()` to limit fields to those needed for serialization
- Cache full unfiltered GeoJSON response in Redis (5-minute TTL); invalidate on SpeciesLocality save/delete signals

Acceptance Criteria:

Given a GET to `/api/v1/map/localities/` with no parameters **When** the database contains 50 locality records **Then** a GeoJSON FeatureCollection with 50 features is returned; each feature has `type: "Point"` geometry and all specified properties

Given a GET to `/api/v1/map/localities/?species_id=7` **When** species 7 has 3 locality records **Then** the response contains exactly 3 features, all with `properties.species_id = 7`

Given a GET to `/api/v1/map/localities/?iucn_status=CR&family=Bedotiidae` **When** multiple filters are applied simultaneously **Then** only localities for CR-status Bedotiidae species are returned (filters are AND-combined)

Given a GET to `/api/v1/map/localities/` and a SpeciesLocality record has `is_sensitive = True` **When** the response is returned **Then** the geometry coordinates for that feature are the generalized coordinates (rounded to 0.1 degree), not the exact coordinates

Given a GET to `/api/v1/map/localities/?bbox=46.0,-20.0,48.0,-18.0` **When** the bbox filter is applied **Then** only localities with coordinates within the bounding box are returned

Given a GET to `/api/v1/map/localities/` and the database has zero SpeciesLocality records **When** the response is returned **Then** HTTP 200 with `{"type": "FeatureCollection", "features": []}` — not a 404

BE-05-8: Watershed List Endpoint

As the Next.js map page, **I want** `GET /api/v1/map/watersheds/` to return a list of watersheds with species counts, **so that** I can populate the watershed filter dropdown and watershed info panels.

Response (200 OK):

```
[
  {
    "id": 1,
    "name": "Betsiboka",
    "pfafstetter_level": 6,
    "area_sq_km": 48900.00,
    "species_count": 12
  }
]
```

`species_count` is annotated via a subquery counting distinct species in SpeciesLocality records for that watershed.

Note: Watershed geometries are NOT served through this endpoint. They are served as static GeoJSON files at `/static/map-layers/watersheds.geojson`. This endpoint provides metadata only.

Acceptance Criteria:

Given a GET to `/api/v1/map/watersheds/` **When** 60 Watershed records exist, 34 of which have at least one SpeciesLocality **Then** all 60 watersheds are returned; watersheds with localities have `species_count > 0`; watersheds without localities have `species_count = 0`

Given a GET to `/api/v1/map/watersheds/` and no Watershed records exist **When** the response is returned **Then** HTTP 200 with an empty array `[]`

BE-05-9: Map Summary Endpoint

As the Next.js map page, **I want** `GET /api/v1/map/summary/` to return aggregate map statistics, **so that** I can render the statistics bar and communicate data coverage.

Response (200 OK):

```
{
  "total_localities": 547,
  "species_with_localities": 72,
  "species_without_localities": 7,
  "watersheds_represented": 34,
  "locality_type_counts": {
    "type_locality": 79,
    "collection_record": 312,
    "literature_record": 98,
    "observation": 58
  },
  "presence_status_counts": {
    "present": 420,
```

```
    "historically_present_extirpated": 67,
    "presence_unknown": 45,
    "reintroduced": 15
  }
}
```

Cached in Redis (5-minute TTL), invalidated on SpeciesLocality changes.

Acceptance Criteria:

Given a GET to `/api/v1/map/summary/` **When** the database contains 547 SpeciesLocality records across 72 species **Then** `total_localities = 547`, `species_with_localities = 72`, `species_without_localities` equals total species minus 72

Given a GET to `/api/v1/map/summary/` and the database has zero SpeciesLocality records **When** the response is returned **Then** `total_localities = 0`, `species_with_localities = 0`, all type/status counts are 0 or omitted; HTTP 200 (not an error)

Technical Tasks

- Create `species/views.py`, `populations/views.py`, `fieldwork/views.py`, `coordination/views.py` (empty for now), `integration/views.py` (health only)
- Create `species/serializers.py` with `SpeciesListSerializer`, `SpeciesDetailSerializer` using `SerializerMethodField` for tier-gated fields
- Create `ConservationAssessmentSerializer` with tier-gated `review_status` / `review_notes` / `flagged_by` / `flagged_date`
- Create `ExSituPopulationSerializer` and `ExSituPopulationDetailSerializer`
- Create `InstitutionSerializer` with tier-gated `contact_email`
- Create `DashboardSerializer` (read-only, aggregated queryset)
- Configure URL routing in `config/urls.py` and per-app `urls.py`
- Generate OpenAPI schema with `drf-spectacular`; serve at `/api/schema/`, Swagger UI at `/api/docs/`
- Write integration tests: one test per tier (anonymous, Tier 2, Tier 3, Tier 5) for each endpoint; verify field presence/absence matches tier rules
- Use `select_related` and `prefetch_related` on all list views to eliminate N+1 queries; verify with `django-assert-num-queries` in tests
- Add `djangorestframework-gis` to `requirements.txt` and `rest_framework_gis` to `INSTALLED_APPS`
- Create `species/serializers_map.py` with `SpeciesLocalityGeoSerializer` (using `GeoFeatureModelSerializer`), `WatershedListSerializer`, `MapSummarySerializer`
- Create `species/views_map.py` with `SpeciesLocalityGeoViewSet`, `WatershedListView`, `MapSummaryView`
- Register URL routes under `/api/v1/map/` namespace in `config/urls.py`
- Write integration tests: GeoJSON response shape validation, filter combinations, sensitive coordinate redaction, empty state responses, N+1 query tests for the localities endpoint

---

Out of Scope

- Write endpoints (POST/PUT/PATCH/DELETE) — all writes go through Django Admin at MVP
- Occurrence record endpoints (post-MVP)

- Breeding recommendation endpoints (post-MVP)
- GBIF publishing endpoints (post-MVP)
- FishBase data fields (post-MVP)

Gate Exit Criteria

Before marking Gate 05 complete:

1. All endpoints return correct HTTP status codes and response shapes per acceptance criteria
2. Tier-gating integration tests pass (field presence/absence verified for all tiers on all endpoints)
3. N+1 query tests pass for all list endpoints
4. OpenAPI schema generates without errors
5. `/api/v1/map/localities/` returns valid GeoJSON FeatureCollection with correct response shape
6. Sensitive record coordinates are redacted (generalized coordinates served for `is_sensitive = True` records)
7. All map filter parameters work correctly (species_id, family, iucn_status, watershed_id, locality_type, presence_status, coordinate_precision, bbox)
8. Map endpoints return HTTP 200 with empty collections (not 404) when no data exists
9. `djangorestframework-gis` is installed and configured
10. Invoke **@test-writer** to write adversarial API tests (tier escalation, malformed filter params, large page sizes, malformed bbox, GeoJSON shape validation)
11. Invoke **@security-reviewer** — this gate defines all data access boundaries
12. Invoke **@code-quality-reviewer** on serializers and views

---

Reconciliation

See gate-05-reconciliation.md for the full spec-vs-implementation audit trail.